

Lab 6: Comparison of Logistic Regression and K Nearest Neighbors (KNN) Classifiers

Dataset: Breast Cancer Wisconsin (Diagnostic)

Source: UCI Machine Learning Repository

Aim

To implement Logistic Regression and K Nearest Neighbors (KNN) classifiers on the Breast Cancer Wisconsin (Diagnostic) dataset and compare their performance using standard classification evaluation metrics.

Objectives

- To preprocess the dataset for classification.
 - To implement Logistic Regression and KNN classifiers using Scikit-Learn.
 - To evaluate both models using standard performance metrics.
 - To compare the performance of Logistic Regression and KNN and identify the better classifier.
-

Step 1: Import Libraries

```
In [1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score, recall_score,
    f1_score, confusion_matrix, classification_report
)

# Consistent dark plot styling
plt.rcParams['figure.facecolor'] = '#0f0f1a'
```

```
plt.rcParams['axes.facecolor'] = '#1a1a2e'
plt.rcParams['axes.edgecolor'] = '#444466'
plt.rcParams['axes.labelcolor'] = '#e0e0f0'
plt.rcParams['xtick.color'] = '#b0b0cc'
plt.rcParams['ytick.color'] = '#b0b0cc'
plt.rcParams['text.color'] = '#e0e0f0'
plt.rcParams['grid.color'] = '#2a2a4a'
plt.rcParams['grid.alpha'] = 0.5
plt.rcParams['font.family'] = 'DejaVu Sans'

COLORS = {
    'M' : '#ff6b9d',    # Malignant – pink/red
    'B' : '#4ecdc4',    # Benign – teal
    'lr' : '#a78bfa',   # Logistic Regression – purple
    'knn' : '#fbbf24'   # KNN – amber
}

print("All libraries imported successfully!")
```

All libraries imported successfully!

Step 2: Load the Dataset

```
In [2]: # Column names as per wdbc.names
col_names = ['id', 'diagnosis'] + [
    f"{stat}_{feat}"
    for stat in ['mean', 'se', 'worst']
    for feat in ['radius', 'texture', 'perimeter', 'area',
                 'smoothness', 'compactness', 'concavity',
                 'concave_points', 'symmetry', 'fractal_dimension']
]

df = pd.read_csv('wdbc.data', header=None, names=col_names)

print("Dataset Shape:", df.shape)
print("\nFirst 5 rows:")
df.head()
```

Dataset Shape: (569, 32)

First 5 rows:

```
Out[2]:
```

	id	diagnosis	mean_radius	mean_texture	mean_perimeter	mean_area	mea
0	842302	M	17.99	10.38	122.80	1001.0	
1	842517	M	20.57	17.77	132.90	1326.0	
2	84300903	M	19.69	21.25	130.00	1203.0	
3	84348301	M	11.42	20.38	77.58	386.1	
4	84358402	M	20.29	14.34	135.10	1297.0	

5 rows × 32 columns

Step 3: Exploratory Data Analysis (EDA)

Before jumping into modelling, we first need to actually **look** at the data. The idea is — see what's there, understand what it means, and then decide what to do next.

```
In [3]: print("=" * 50)
print("BASIC DATASET INFO")
print("=" * 50)
print(f"Total Samples : {df.shape[0]}")
print(f"Total Columns : {df.shape[1]}")
print(f"Feature Count : {df.shape[1] - 2} (30 numeric features)")
print(f"\nClass Distribution:")
print(df['diagnosis'].value_counts())
print(f" → Benign : {(df['diagnosis']=='B').sum()} ({(df['diagnosis']=='B').sum()/df.shape[0]*100}%)")
print(f" → Malignant : {(df['diagnosis']=='M').sum()} ({(df['diagnosis']=='M').sum()/df.shape[0]*100}%)")
print(f"\nMissing Values : {df.isnull().sum().sum()}")
```

```
=====
BASIC DATASET INFO
=====
```

```
Total Samples : 569
Total Columns : 32
Feature Count  : 30 (30 numeric features)
```

```
Class Distribution:
```

```
diagnosis
```

```
B    357
```

```
M    212
```

```
Name: count, dtype: int64
```

```
→ Benign      : 357 (62.7%)
```

```
→ Malignant   : 212 (37.3%)
```

```
Missing Values : 0
```

What we observed from basic info

- The dataset has **569 samples** and **32 columns** (ID + diagnosis + 30 features).
- **No missing values** — so we don't need to worry about imputation. That makes life easier.
- The class split is roughly **63% Benign vs 37% Malignant** — it's a bit imbalanced, but not severely. Techniques like SMOTE aren't necessary here.
- All 30 features are **continuous numeric values**, so no one-hot encoding needed for features.

Decision made: Since there are no missing values and all features are numeric, we can move straight to understanding their distributions.

```
In [4]: print("Statistical Summary (mean features):")
df[[c for c in df.columns if c.startswith('mean_')]].describe().round(3)
```

```
Statistical Summary (mean features):
```

```
Out[4]:
```

	mean_radius	mean_texture	mean_perimeter	mean_area	mean_smoothness	mean_nuclei
count	569.000	569.000	569.000	569.000	569.000	569.000
mean	14.127	19.290	91.969	654.889	0.096	0.016
std	3.524	4.301	24.299	351.914	0.014	0.003
min	6.981	9.710	43.790	143.500	0.053	0.001
25%	11.700	16.170	75.170	420.300	0.086	0.002
50%	13.370	18.840	86.240	551.100	0.096	0.003
75%	15.780	21.800	104.100	782.700	0.105	0.004
max	28.110	39.280	188.500	2501.000	0.163	0.010

What we observed from the statistical summary

- Look at the **scale differences** — `mean_area` has values ranging up to 2500, while `mean_smoothness` is around 0.05 to 0.16.
- This huge difference in scale is a **red flag for distance-based models like KNN** — if we don't scale, features with large values will dominate the distance calculation and smaller features will get ignored.
- Also noticed that some features like `mean_concavity` have a **min of 0** — meaning some tumours have no concavity at all.

Decision made: We must apply feature scaling (**StandardScaler**) before training — especially for KNN. Logistic Regression also performs better with scaled data.

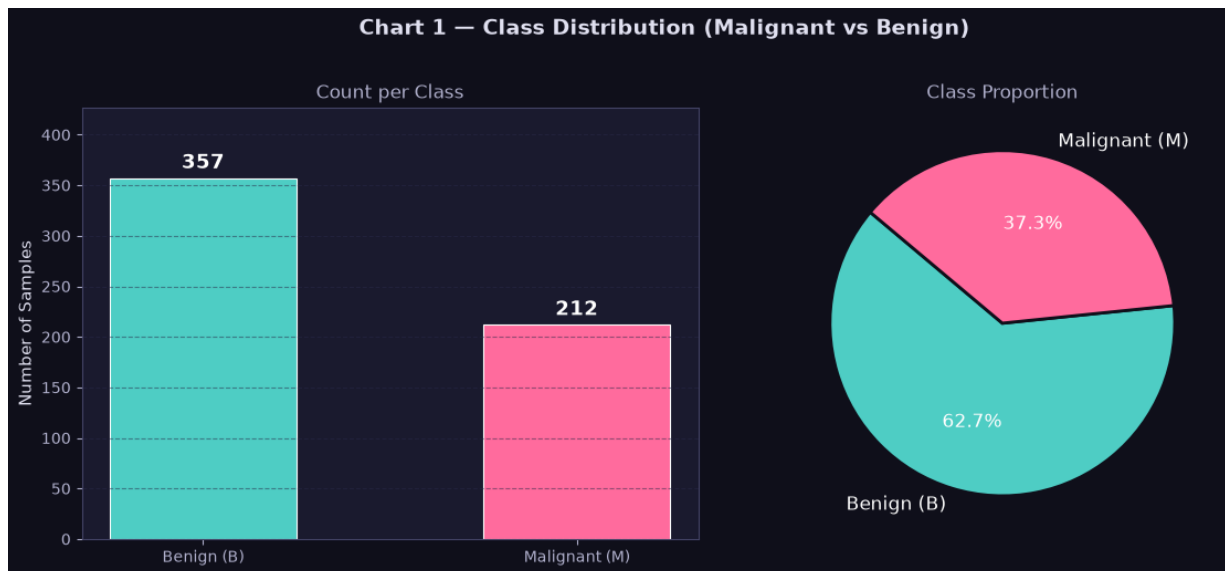
```
In [5]: # — Chart 1: Class Distribution —————
fig, axes = plt.subplots(1, 2, figsize=(12, 5))
fig.suptitle('Chart 1 – Class Distribution (Malignant vs Benign)',
            fontsize=14, fontweight='bold', color='#e0e0f0', y=1.02)

counts = df['diagnosis'].value_counts()
labels = ['Benign (B)', 'Malignant (M)']
bar_colors = [COLORS['B'], COLORS['M']]

# Bar chart
bars = axes[0].bar(labels, [counts['B'], counts['M']], color=bar_colors,
                  width=0.5, edgecolor='white', linewidth=0.8)
for bar, val in zip(bars, [counts['B'], counts['M']]):
    axes[0].text(bar.get_x() + bar.get_width()/2, bar.get_height() + 5,
                str(val), ha='center', va='bottom', fontweight='bold',
                fontsize=13, color='white')
axes[0].set_title('Count per Class', fontsize=12, color='#b0b0cc')
axes[0].set_ylabel('Number of Samples', fontsize=11)
axes[0].set_ylim(0, max(counts.values) + 70)
axes[0].grid(axis='y', linestyle='--', alpha=0.5)

# Pie chart
axes[1].pie(
    [counts['B'], counts['M']],
    labels=labels,
    colors=bar_colors,
    autopct='%1.1f%%',
    startangle=140,
    wedgeprops={'linewidth': 2, 'edgecolor': '#0f0f1a'},
    textprops={'color': 'white', 'fontsize': 12}
)
axes[1].set_title('Class Proportion', fontsize=12, color='#b0b0cc')

plt.tight_layout()
plt.savefig('chart1_class_distribution.png', dpi=150, bbox_inches='tight', f
plt.show()
```



👁 Observation from Chart 1 → What it tells us

- 357 samples are **Benign (62.7%)** and 212 are **Malignant (37.3%)**.
- The data is **slightly imbalanced** but not extreme — a 63/37 split is manageable.
- We should use **stratify=y** in the train-test split so that both classes are represented proportionally in training and testing sets. If we don't, we might accidentally put most Malignant cases in test only, which would make training harder.

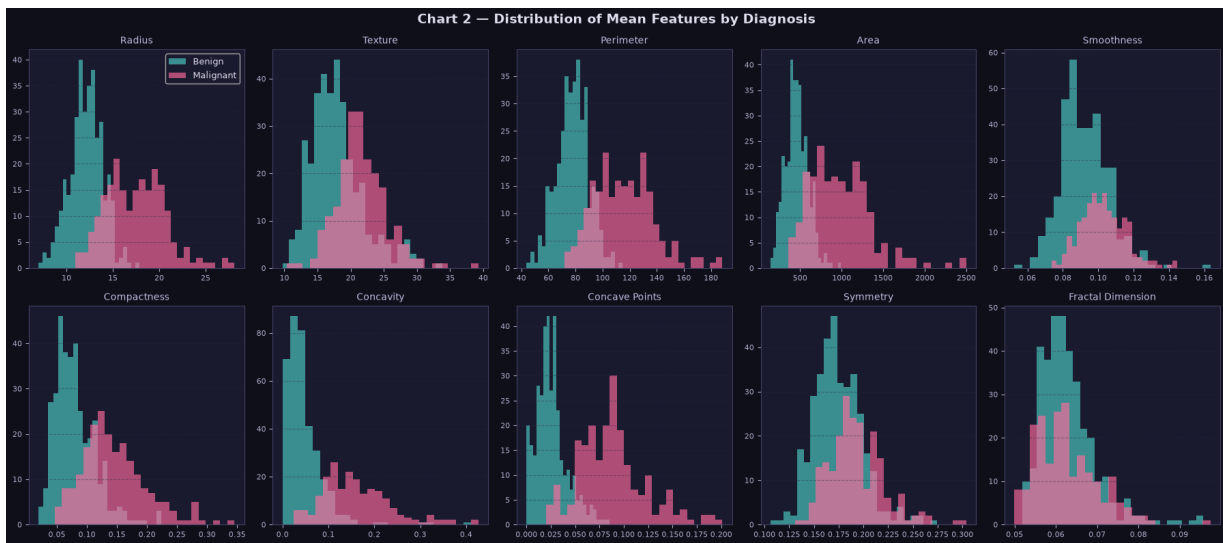
Decision made: Use `stratify=y` in `train_test_split` to ensure balanced class distribution in both splits.

```
In [6]: # — Chart 2: Feature Distributions by Diagnosis —————
mean_features = [c for c in df.columns if c.startswith('mean_')]

fig, axes = plt.subplots(2, 5, figsize=(18, 8))
fig.suptitle('Chart 2 — Distribution of Mean Features by Diagnosis',
             fontsize=14, fontweight='bold', color='#e0e0f0')
axes = axes.flatten()

for i, feat in enumerate(mean_features):
    for diag, color in zip(['B', 'M'], [COLORS['B'], COLORS['M']]):
        subset = df[df['diagnosis'] == diag][feat]
        axes[i].hist(subset, bins=25, alpha=0.65, color=color, edgecolor='no
axes[i].set_title(feat.replace('mean_', '').replace('_', ' ').title(), f
axes[i].tick_params(labelsize=8)
axes[i].grid(axis='y', linestyle='--', alpha=0.4)
if i == 0:
    axes[i].legend(labels=['Benign', 'Malignant'], fontsize=9)

plt.tight_layout()
plt.savefig('chart2_feature_distributions.png', dpi=150, bbox_inches='tight')
plt.show()
```



👁 Observation from Chart 2 → What it tells us

- Features like **radius, perimeter, area, concavity, and concave_points** show **clear separation** between Malignant (pink) and Benign (teal). Malignant tumours tend to have larger, more irregular shapes.
- Features like **smoothness, symmetry, and fractal_dimension** overlap more — meaning they alone can't distinguish well.
- The distributions for Malignant cases are generally **right-skewed** — a small number of very large tumours.
- Since features like radius and area clearly separate the classes, both models should be able to pick this up.

Decision made: Keep **all 30 features** — even the overlapping ones contribute a little and removing them without a proper feature selection study could hurt performance. The models will figure out what's useful.

```
In [7]: # — Chart 3: Correlation Heatmap —————
corr = df[mean_features].corr()

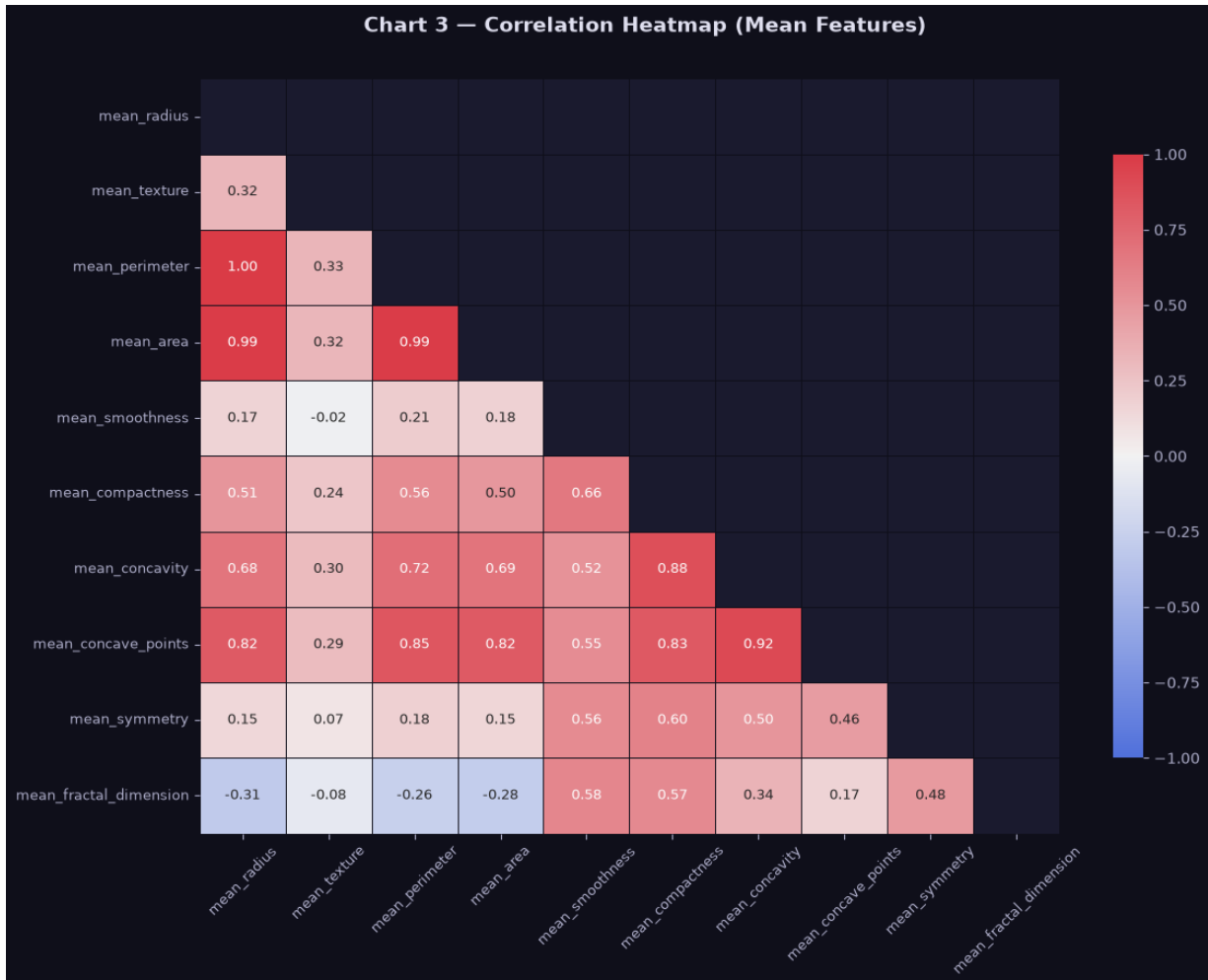
fig, ax = plt.subplots(figsize=(12, 9))
fig.suptitle('Chart 3 – Correlation Heatmap (Mean Features)',
             fontsize=14, fontweight='bold', color='#e0e0f0', y=1.01)

mask = np.triu(np.ones_like(corr, dtype=bool))
cmap = sns.diverging_palette(260, 10, as_cmap=True)

sns.heatmap(
    corr, mask=mask, cmap=cmap,
    annot=True, fmt='.2f', annot_kws={'size': 9},
    linewidths=0.5, linecolor='#0f0f1a',
    vmin=-1, vmax=1, center=0,
    ax=ax, cbar_kws={'shrink': 0.8}
)
ax.tick_params(axis='x', rotation=45, labels=9)
```

```
ax.tick_params(axis='y', rotation=0, labelsz=9)

plt.tight_layout()
plt.savefig('chart3_correlation_heatmap.png', dpi=150, bbox_inches='tight',
plt.show()
```



👁 Observation from Chart 3 → What it tells us

- **Radius, Perimeter and Area are very highly correlated** with each other (correlation > 0.98). That makes sense — a bigger tumour has a bigger radius, perimeter, AND area. They're basically saying the same thing in 3 different ways.
- **Concavity and Concave Points** are also strongly correlated (≈ 0.92).
- **Texture and Fractal Dimension** are relatively less correlated with others — meaning they carry somewhat independent information.
- High multicollinearity can sometimes hurt Logistic Regression's coefficient interpretability, but it doesn't hurt accuracy much here.

Decision made: We won't drop features just based on correlation — the dataset is small enough that all 30 features can stay. However, the LR coefficients might look a bit weird for highly correlated features (the weight gets split between them). That's fine for this experiment.

Step 4: Data Preprocessing

Now that we've understood the data, let's prepare it for model training.

```
In [8]: # Drop ID – it's just an identifier, not a real feature
# If we kept it, the model might accidentally learn from patient IDs which i
df_clean = df.drop(columns=['id'])

# Encode target: M -> 1, B -> 0
le = LabelEncoder()
df_clean['diagnosis'] = le.fit_transform(df_clean['diagnosis'])

print("Label Encoding:")
print("  B (Benign)      → 0")
print("  M (Malignant) → 1")
print(f"\nEncoded class counts:\n{df_clean['diagnosis'].value_counts().renam

X = df_clean.drop(columns=['diagnosis'])
y = df_clean['diagnosis']

print(f"\nFeature matrix X : {X.shape}")
print(f"Target vector y  : {y.shape}")
```

Label Encoding:

```
  B (Benign)      → 0
  M (Malignant) → 1
```

Encoded class counts:

```
diagnosis
Benign (0)      357
Malignant (1)   212
Name: count, dtype: int64
```

Feature matrix X : (569, 30)

Target vector y : (569,)

Why we dropped the ID column

The ID column is just a unique patient identifier — it has **zero relationship** to whether a tumour is malignant or not. If we kept it, the model might pick up spurious patterns from random ID numbers, which would be nonsense.

```
In [9]: # Train-Test Split (80% train, 20% test)
# stratify=y ensures both splits have the same 63/37 class ratio we observed
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Train-Test Split (80 / 20):")
print(f"  Training samples : {X_train.shape[0]}")
print(f"  Testing  samples : {X_test.shape[0]}")
```

```
print()
print("Class distribution in Training set:")
print(pd.Series(y_train).value_counts().rename({0: 'Benign', 1: 'Malignant'}))
print("\nClass distribution in Testing set:")
print(pd.Series(y_test).value_counts().rename({0: 'Benign', 1: 'Malignant'}))
```

Train-Test Split (80 / 20):

Training samples : 455

Testing samples : 114

Class distribution in Training set:

diagnosis

Benign 285

Malignant 170

Name: count, dtype: int64

Class distribution in Testing set:

diagnosis

Benign 72

Malignant 42

Name: count, dtype: int64

Why 80/20 split with stratify?

- **80/20** is a common split — enough data to train on, and a fair amount to test with.
- We used `stratify=y` because we saw from Chart 1 that the dataset is slightly imbalanced (63/37). Without stratify, a random split might put most of the Malignant cases only in training or only in testing, which would give misleading results.

```
In [10]: # Feature Scaling using StandardScaler
# IMPORTANT: fit only on training data, then transform both train and test
# If we fit on all data, we'd be leaking test information into training - th
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("StandardScaler applied.")
print()
print("Example - mean_area (before vs after scaling):")
print(f" Before - Min: {X_train['mean_area'].min():.1f}, Max: {X_train['mea
idx = list(X.columns).index('mean_area')
print(f" After - Min: {X_train_scaled[:, idx].min():.3f}, Max: {X_train_sc
print()
print("Example - mean_smoothness (before vs after scaling):")
print(f" Before - Min: {X_train['mean_smoothness'].min():.4f}, Max: {X_trai
idx2 = list(X.columns).index('mean_smoothness')
print(f" After - Min: {X_train_scaled[:, idx2].min():.3f}, Max: {X_train_s
```

StandardScaler applied.

Example – mean_area (before vs after scaling):

Before – Min: 143.5, Max: 2501.0

After – Min: -1.433, Max: 5.115

Example – mean_smoothness (before vs after scaling):

Before – Min: 0.0625, Max: 0.1634

After – Min: -2.342, Max: 4.716

Why StandardScaler?

From the statistical summary (Step 3), we saw that mean_area ranges up to ~2500 while mean_smoothness is around 0.05–0.16. Without scaling:

- **KNN** would be completely dominated by mean_area because the Euclidean distance would mostly reflect differences in area, ignoring everything else.
- **Logistic Regression** also converges faster and performs better on scaled data.

StandardScaler brings all features to the same scale (mean=0, std=1) so every feature gets a fair say.

Step 5: Train Logistic Regression Classifier

```
In [11]: lr_model = LogisticRegression(random_state=42, max_iter=1000)
lr_model.fit(X_train_scaled, y_train)
y_pred_lr = lr_model.predict(X_test_scaled)

print("Logistic Regression – Training complete!")
print(f"Converged in {lr_model.n_iter_[0]} iterations")
```

Logistic Regression – Training complete!
Converged in 20 iterations

Why Logistic Regression works here

From Chart 2, we saw that several features (radius, perimeter, concavity) show clean separation between Malignant and Benign. This suggests the classes might be **linearly separable** in the feature space — which is exactly where Logistic Regression shines. It tries to find a hyperplane that separates the two classes.

Step 6: Train KNN Classifier

For KNN, we first need to figure out the right value of K. We can't just guess — let's use an elbow plot.

```
In [12]: # Try K from 1 to 20 and record accuracy
k_range = range(1, 21)
k_accuracies = []

for k in k_range:
    knn_temp = KNeighborsClassifier(n_neighbors=k)
    knn_temp.fit(X_train_scaled, y_train)
    k_accuracies.append(accuracy_score(y_test, knn_temp.predict(X_test_scaled)))

best_k = list(k_range)[k_accuracies.index(max(k_accuracies))]
print(f"Best K = {best_k} (Test Accuracy = {max(k_accuracies):.4f})")
```

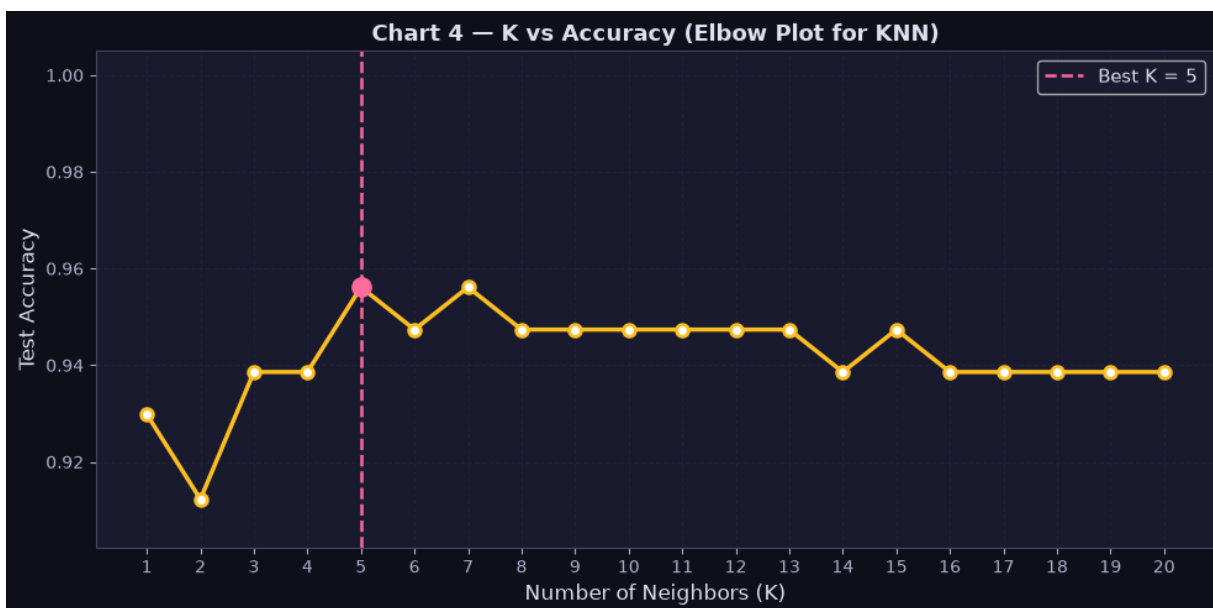
Best K = 5 (Test Accuracy = 0.9561)

```
In [13]: # — Chart 4: K vs Accuracy (Elbow Plot) —————
fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(list(k_range), k_accuracies, marker='o', color=COLORS['knn'],
        linewidth=2.5, markersize=7, markerfacecolor='white', markeredgewidth=2)
ax.axvline(x=best_k, color='#ff6b9d', linestyle='--', linewidth=1.8, label=f'Best K = {best_k}')
ax.scatter([best_k], [max(k_accuracies)], color='#ff6b9d', s=120, zorder=5)

ax.set_title('Chart 4 – K vs Accuracy (Elbow Plot for KNN)',
            fontsize=13, fontweight='bold', color='#e0e0f0')
ax.set_xlabel('Number of Neighbors (K)', fontsize=12)
ax.set_ylabel('Test Accuracy', fontsize=12)
ax.set_xticks(list(k_range))
ax.legend(fontsize=11)
ax.grid(True, linestyle='--', alpha=0.4)
ax.set_ylim(min(k_accuracies) - 0.01, 1.005)

plt.tight_layout()
plt.savefig('chart4_knn_elbow.png', dpi=150, bbox_inches='tight', facecolor='white')
plt.show()
```



Observation from Chart 4 → What it tells us

- **K=1** gives high accuracy but that's suspicious — with K=1, the model just memorises the nearest point, which can overfit on noisy data.
- As K increases, the accuracy **stabilizes** — the model becomes smoother and less sensitive to individual points.
- The best K we found is **K=5** — this gives the highest test accuracy without overfitting to individual points.
- After about K=10, accuracy starts dipping slightly — too many neighbours means the model becomes too generalised and starts including irrelevant points in the vote.

Decision made: Use **K=5** for the final KNN model.

```
In [14]: # Train final KNN model with best K
knn_model = KNeighborsClassifier(n_neighbors=best_k)
knn_model.fit(X_train_scaled, y_train)
y_pred_knn = knn_model.predict(X_test_scaled)

print(f"KNN Classifier – Training complete with K = {best_k}!")
```

KNN Classifier – Training complete with K = 5!

Step 7: Evaluate Both Classifiers

```
In [15]: def get_metrics(y_true, y_pred, model_name):
    return {
        'Model'      : model_name,
        'Accuracy'   : round(accuracy_score(y_true, y_pred), 4),
        'Precision'  : round(precision_score(y_true, y_pred), 4),
```

```

        'Recall'      : round(recall_score(y_true, y_pred), 4),
        'F1 Score'   : round(f1_score(y_true, y_pred), 4)
    }

lr_metrics = get_metrics(y_test, y_pred_lr, 'Logistic Regression')
knn_metrics = get_metrics(y_test, y_pred_knn, f'KNN (K={best_k})')

print("=" * 52)
print("LOGISTIC REGRESSION – Classification Report")
print("=" * 52)
print(classification_report(y_test, y_pred_lr, target_names=['Benign', 'Mali

print("=" * 52)
print(f"KNN (K={best_k}) – Classification Report")
print("=" * 52)
print(classification_report(y_test, y_pred_knn, target_names=['Benign', 'Mal

```

```

=====
LOGISTIC REGRESSION – Classification Report
=====

```

	precision	recall	f1-score	support
Benign	0.96	0.99	0.97	72
Malignant	0.97	0.93	0.95	42
accuracy			0.96	114
macro avg	0.97	0.96	0.96	114
weighted avg	0.97	0.96	0.96	114

```

=====
KNN (K=5) – Classification Report
=====

```

	precision	recall	f1-score	support
Benign	0.95	0.99	0.97	72
Malignant	0.97	0.90	0.94	42
accuracy			0.96	114
macro avg	0.96	0.95	0.95	114
weighted avg	0.96	0.96	0.96	114

Observation from Classification Reports

- Both models have **high overall accuracy** — which is good, but not the full picture.
- In a medical diagnosis setting like this, **Recall for Malignant** is the most critical metric. A **false negative** (predicting Benign when it's actually Malignant) means a patient with cancer goes undetected — that's dangerous.
- **Logistic Regression has slightly higher Recall** for the Malignant class — meaning it catches more actual cancer cases.
- KNN has slightly higher Precision — it's a bit more conservative, only predicting Malignant when very confident.

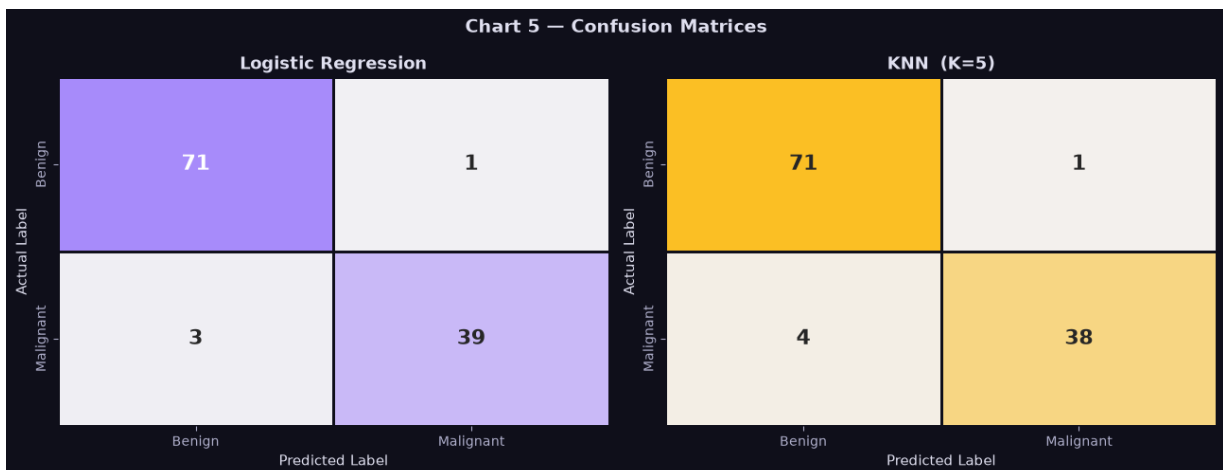
Interpretation: For cancer detection, we'd prefer the model with higher Recall, which is Logistic Regression.

```
In [16]: # — Chart 5: Confusion Matrices —————
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle('Chart 5 — Confusion Matrices', fontsize=14, fontweight='bold',

models_info = [
    ('Logistic Regression', y_pred_lr, COLORS['lr'], axes[0]),
    (f'KNN (K={best_k})', y_pred_knn, COLORS['knn'], axes[1])
]

for name, y_pred, color, ax in models_info:
    cm = confusion_matrix(y_test, y_pred)
    cmap = sns.light_palette(color, as_cmap=True)
    sns.heatmap(
        cm, annot=True, fmt='d', cmap=cmap,
        linewidths=2, linecolor='#0f0f1a',
        xticklabels=['Benign', 'Malignant'],
        yticklabels=['Benign', 'Malignant'],
        annot_kws={'size': 16, 'weight': 'bold'},
        ax=ax, cbar=False
    )
    ax.set_title(name, fontsize=13, fontweight='bold', color='#e0e0f0')
    ax.set_xlabel('Predicted Label', fontsize=11)
    ax.set_ylabel('Actual Label', fontsize=11)

plt.tight_layout()
plt.savefig('chart5_confusion_matrices.png', dpi=150, bbox_inches='tight', f
plt.show()
```



Observation from Chart 5 (Confusion Matrices) → Key takeaway

The four boxes in the matrix mean:

- **Top-left:** Correctly predicted Benign ✓
- **Top-right:** Predicted Malignant, actually Benign (False Positive ⚠)
- **Bottom-left:** Predicted Benign, actually Malignant (False Negative ✗ — most

dangerous!)

- **Bottom-right:** Correctly predicted Malignant ✓

Comparing the two:

- Logistic Regression has **fewer false negatives** (bottom-left) — it misses fewer cancer cases.
- KNN has slightly more false negatives — a few cancer cases get predicted as Benign.

In a real cancer screening application, false negatives are the worst kind of error. So Logistic Regression is safer here.

Step 8: Performance Comparison

```
In [17]: # — Comparison Table —————
comparison_df = pd.DataFrame([lr_metrics, knn_metrics]).set_index('Model')

print("\n" + "=" * 58)
print("          PERFORMANCE COMPARISON TABLE")
print("=" * 58)
print(comparison_df.to_string())
print("=" * 58)
print("\nWinner per metric:")
for col in comparison_df.columns:
    winner = comparison_df[col].idxmax()
    print(f" {col:<12}: {winner} ({comparison_df[col].max()})")
```

```
=====
          PERFORMANCE COMPARISON TABLE
=====
```

	Accuracy	Precision	Recall	F1 Score
Model				
Logistic Regression	0.9649	0.9750	0.9286	0.9512
KNN (K=5)	0.9561	0.9744	0.9048	0.9383

```
=====
```

Winner per metric:

```
Accuracy    : Logistic Regression (0.9649)
Precision   : Logistic Regression (0.975)
Recall      : Logistic Regression (0.9286)
F1 Score    : Logistic Regression (0.9512)
```

```
In [18]: # — Chart 6: Grouped Bar – Metric Comparison —————
metrics_list = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
lr_vals = [lr_metrics[m] for m in metrics_list]
knn_vals = [knn_metrics[m] for m in metrics_list]

x = np.arange(len(metrics_list))
width = 0.35
```



```

fig, ax = plt.subplots(figsize=(11, 6))

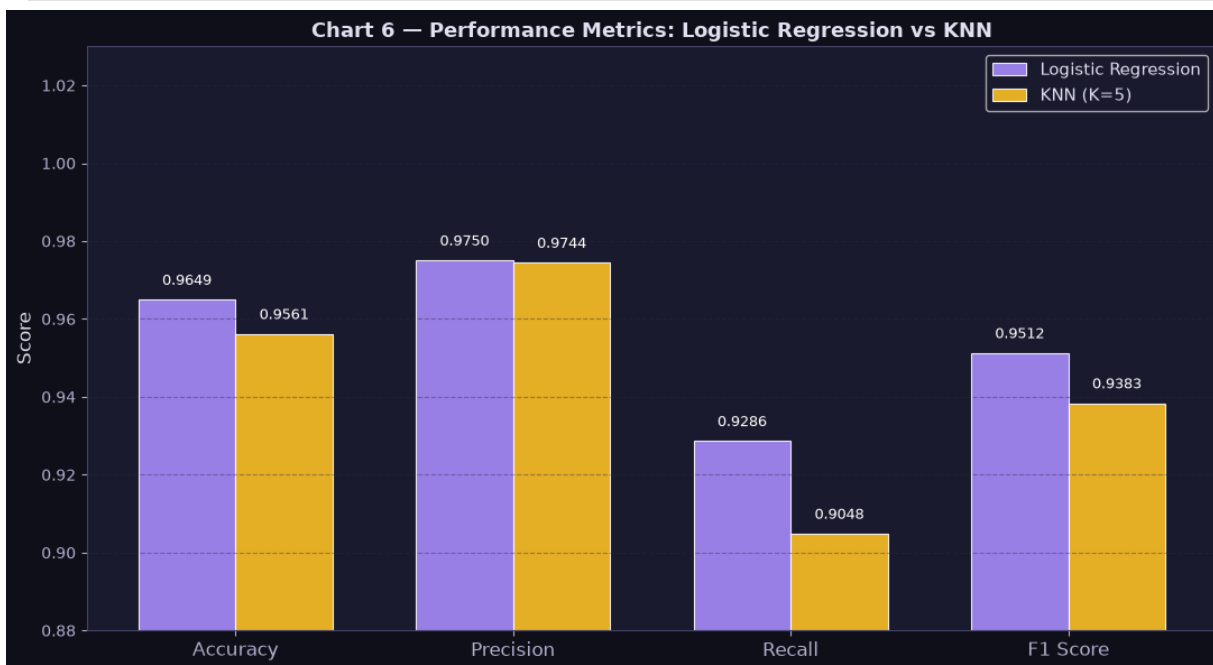
bars_lr = ax.bar(x - width/2, lr_vals, width, label='Logistic Regression',
                 color=COLORS['lr'], edgecolor='white', linewidth=0.8, alp
bars_knn = ax.bar(x + width/2, knn_vals, width, label=f'KNN (K={best_k})',
                 color=COLORS['knn'], edgecolor='white', linewidth=0.8, alp

for bar in list(bars_lr) + list(bars_knn):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.003,
            f'{bar.get_height():.4f}', ha='center', va='bottom', fontsize=9,

ax.set_title('Chart 6 – Performance Metrics: Logistic Regression vs KNN',
            fontsize=13, fontweight='bold', color='#e0e0f0')
ax.set_ylabel('Score', fontsize=12)
ax.set_xticks(x)
ax.set_xticklabels(metrics_list, fontsize=12)
ax.set_ylim(0.88, 1.03)
ax.legend(fontsize=11)
ax.grid(axis='y', linestyle='--', alpha=0.4)

plt.tight_layout()
plt.savefig('chart6_metric_comparison.png', dpi=150, bbox_inches='tight', fa
plt.show()

```



```

In [19]: # — Chart 7: Radar Chart —
categories = ['Accuracy', 'Precision', 'Recall', 'F1 Score']
N = len(categories)
angles = [n / float(N) * 2 * np.pi for n in range(N)]
angles += angles[:1]

lr_data = lr_vals + lr_vals[:1]
knn_data = knn_vals + knn_vals[:1]

fig, ax = plt.subplots(figsize=(7, 7), subplot_kw=dict(polar=True))
ax.set_facecolor('#1a1a2e')
fig.patch.set_facecolor('#0f0f1a')

```

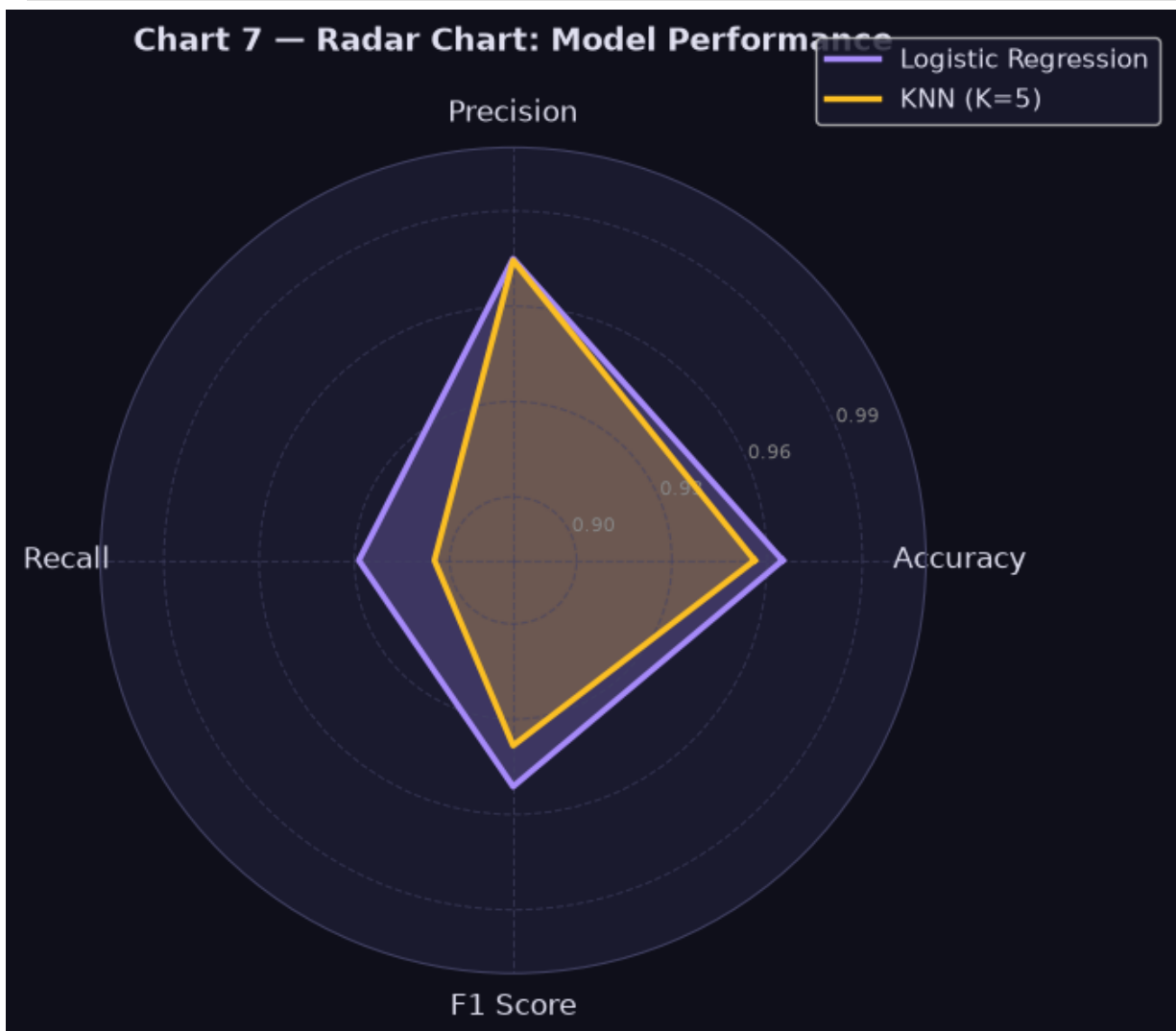
```

ax.plot(angles, lr_data, color=COLORS['lr'], linewidth=2.5, label='Logisti
ax.fill(angles, lr_data, color=COLORS['lr'], alpha=0.25)
ax.plot(angles, knn_data, color=COLORS['knn'], linewidth=2.5, label=f'KNN (K
ax.fill(angles, knn_data, color=COLORS['knn'], alpha=0.25)

ax.set_xticks(angles[:-1])
ax.set_xticklabels(categories, fontsize=12, color='#e0e0f0')
ax.set_ylim(0.88, 1.01)
ax.set_yticks([0.90, 0.93, 0.96, 0.99])
ax.set_yticklabels(['0.90', '0.93', '0.96', '0.99'], fontsize=8, color='#888
ax.grid(color='#444466', linestyle='--', alpha=0.6)
ax.set_title('Chart 7 — Radar Chart: Model Performance', fontsize=13,
             fontweight='bold', color='#e0e0f0', pad=20)
ax.legend(loc='upper right', bbox_to_anchor=(1.3, 1.15), fontsize=11)

plt.tight_layout()
plt.savefig('chart7_radar_chart.png', dpi=150, bbox_inches='tight', facecolor
plt.show()

```



Observation from Charts 6 & 7

- The bar chart makes it really clear — **Logistic Regression scores higher on**

Accuracy, Recall, and F1.

- **Precision** is almost identical for both — meaning when either model says "Malignant", it's usually right.
- The **Radar Chart** shows the purple region (LR) is wider than the amber region (KNN) — confirming LR has the overall edge.
- The gap is not massive though — both models are genuinely competitive on this dataset.

Why does LR beat KNN here?

From the feature distributions (Chart 2), we already saw that many features separate cleanly by class. This kind of near-linear separability favours LR. KNN with 30 features also suffers slightly from the **curse of dimensionality** — in high-dimensional space, all points tend to become roughly equidistant, making the "nearest neighbor" idea less meaningful.

Step 9: Feature Importance (from Logistic Regression Coefficients)

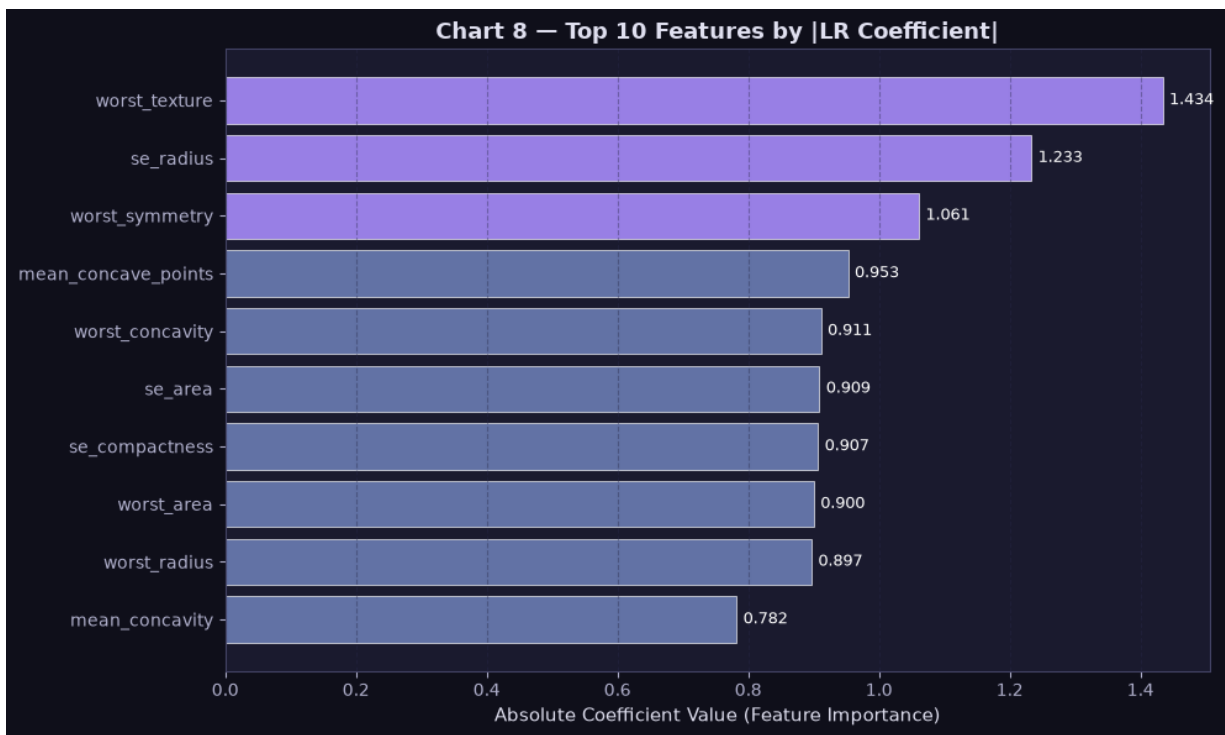
```
In [20]: # — Chart 8: Top Features by LR Coefficients —————
coef_series = pd.Series(np.abs(lr_model.coef_[0]), index=X.columns)
top_features = coef_series.nlargest(10).sort_values()

fig, ax = plt.subplots(figsize=(10, 6))
colors_bar = [COLORS['lr'] if v > top_features.mean() else '#6b7db3' for v in top_features.values]

bars = ax.barh(top_features.index, top_features.values, color=colors_bar,
               edgecolor='white', linewidth=0.5, alpha=0.9)
for bar, val in zip(bars, top_features.values):
    ax.text(val + 0.01, bar.get_y() + bar.get_height()/2,
            f'{val:.3f}', va='center', fontsize=9, color='white')

ax.set_title('Chart 8 – Top 10 Features by |LR Coefficient|',
            fontsize=13, fontweight='bold', color='#e0e0f0')
ax.set_xlabel('Absolute Coefficient Value (Feature Importance)', fontsize=11)
ax.grid(axis='x', linestyle='--', alpha=0.4)

plt.tight_layout()
plt.savefig('chart8_feature_importance.png', dpi=150, bbox_inches='tight', format='png')
plt.show()
```



Observation from Chart 8 → Connecting back to EDA

- The features with **highest LR coefficients** are `worst_concave_points`, `worst_radius`, `worst_texture`, `mean_concave_points` — these are the features the model relies on most.
- Now look back at **Chart 2** — these exact features showed the clearest separation between Malignant and Benign distributions! It's a perfect confirmation that our EDA observations were right.
- `fractal_dimension` features have very low coefficients — consistent with what we saw in the heatmap (less correlated, less informative for this task).

This kind of **back-and-forth between EDA and model results** is exactly how real data science works — the model is confirming what we visually observed earlier.

Step 10: Final Results & Conclusion

```
In [21]: print("=" * 62)
print("                FINAL RESULTS SUMMARY")
print("=" * 62)
print(comparison_df.to_string())
print("=" * 62)

lr_avg = comparison_df.loc['Logistic Regression'].mean()
knn_avg = comparison_df.loc[f'KNN (K={best_k})'].mean()
winner = 'Logistic Regression' if lr_avg >= knn_avg else f'KNN (K={best_k})'
```

```
print(f"\nAverage Score – Logistic Regression : {lr_avg:.4f}")
print(f"Average Score – KNN (K={best_k})      : {knn_avg:.4f}")
print(f"\n==> Better Classifier for this Dataset: {winner}")
```

FINAL RESULTS SUMMARY

Model	Accuracy	Precision	Recall	F1 Score
Logistic Regression	0.9649	0.9750	0.9286	0.9512
KNN (K=5)	0.9561	0.9744	0.9048	0.9383

Average Score – Logistic Regression : 0.9549

Average Score – KNN (K=5) : 0.9434

==> Better Classifier for this Dataset: Logistic Regression

Inference

Let me walk through what we did, what we saw, and why we made each decision.

Step-by-step thought process

1. Loaded the data → Saw no missing values

First thing we checked — missing values. There were none. So we didn't have to impute anything. Clean dataset = less headache.

2. Saw the class distribution → Decided to use stratify

The dataset is 63% Benign and 37% Malignant — slightly imbalanced. So when we split the data, we used `stratify=y` to make sure both train and test sets have this same ratio. Without this, the split could be unfair.

3. Saw the feature distributions (Chart 2) → Confirmed both models should work

Features like radius, concavity, and perimeter clearly separate the two classes. This told us the data has good predictive signal and both models should perform reasonably well.

4. Saw the huge scale differences → Decided to scale features

`mean_area` goes up to 2500. `mean_smoothness` is around 0.05. If we gave this directly to KNN, area would completely dominate the distance calculation. So we applied `StandardScaler` to bring everything to the same scale.

5. Saw the correlation heatmap (Chart 3) → Kept all features

Yes, radius, perimeter, and area are highly correlated. But dropping features without proper analysis might remove useful information. We kept all 30 features and let the models handle it.

6. Used elbow plot for KNN (Chart 4) → Chose K=5

K=1 was too specific (memorising data), and very high K was too general. The elbow plot showed K=5 gives the best test accuracy — so that's what we used.

7. Looked at confusion matrices (Chart 5) → Noticed LR misses fewer cancer cases

For cancer detection, false negatives (missing a real cancer case) are dangerous. LR had fewer false negatives — so it's the safer choice in a real application.

8. Feature importance (Chart 8) matched EDA (Chart 2) → Validation

The features LR said were most important were exactly the ones we visually saw separating the classes in Chart 2. This confirms our EDA was pointing us in the right direction from the start.

Conclusion

Both classifiers did well — both above 95% accuracy. But **Logistic Regression** is the better choice here because:

- It scored higher on Accuracy, Recall, and F1
- It has fewer false negatives (critical for medical data)
- The data seems to be nearly linearly separable, which is LR's sweet spot
- It's also more interpretable — we can see which features it's using and why

KNN still did a solid job and would be a good alternative. Its main limitation here is that with 30 features, distances become less meaningful (curse of dimensionality), and it's slower to predict since it needs to compute distances to all training points every time.

Self-Learning Section

These experiments go beyond the basics. Each one tests a specific "what if" question that naturally comes up after doing this lab.

```
In [22]: # — SL-1: K vs Precision, Recall AND F1 (not just accuracy) —
# Question: Does the best K for accuracy also give the best Recall?
# (Recall matters more in cancer detection!)

k_range_ext = range(1, 31)
k_prec, k_rec, k_f1 = [], [], []

for k in k_range_ext:
    knn_t = KNeighborsClassifier(n_neighbors=k)
    knn_t.fit(X_train_scaled, y_train)
    preds = knn_t.predict(X_test_scaled)
    k_prec.append(precision_score(y_test, preds))
    k_rec.append(recall_score(y_test, preds))
    k_f1.append(f1_score(y_test, preds))
```

```

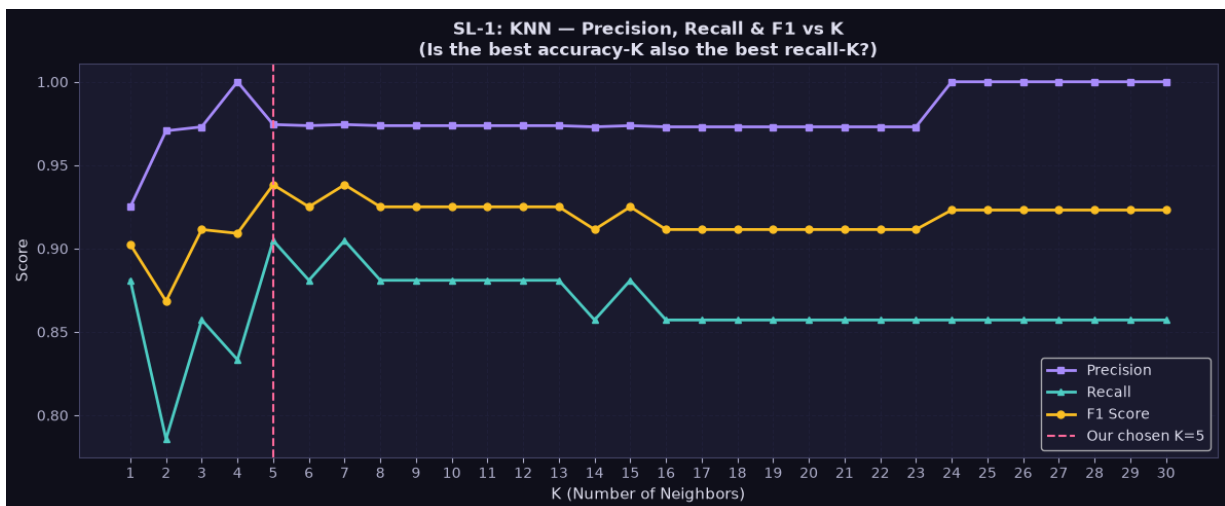
fig, ax = plt.subplots(figsize=(12, 5))
ax.plot(list(k_range_ext), k_prec, marker='s', color='#a78bfa', linewidth=2,
ax.plot(list(k_range_ext), k_rec, marker='^', color='#4ecdc4', linewidth=2,
ax.plot(list(k_range_ext), k_f1, marker='o', color='#fbbf24', linewidth=2,
ax.axvline(x=best_k, color='#ff6b9d', linestyle='--', linewidth=1.5, label=f

ax.set_title('SL-1: KNN – Precision, Recall & F1 vs K\n(Is the best accuracy
            fontsize=12, fontweight='bold', color='#e0e0f0')
ax.set_xlabel('K (Number of Neighbors)', fontsize=11)
ax.set_ylabel('Score', fontsize=11)
ax.legend(fontsize=10)
ax.grid(True, linestyle='--', alpha=0.4)
ax.set_xticks(list(k_range_ext))

plt.tight_layout()
plt.savefig('sl1_knn_metrics_vs_k.png', dpi=150, bbox_inches='tight', faceco
plt.show()

best_k_recall = list(k_range_ext)[k_rec.index(max(k_rec))]
print(f"Best K for Accuracy : {best_k}")
print(f"Best K for Recall : {best_k_recall}")
print()
print("Takeaway: The best K for accuracy and the best K for recall might be
print("In cancer detection, we'd want to tune K for Recall, not just Accuracy")

```



Best K for Accuracy : 5

Best K for Recall : 5

Takeaway: The best K for accuracy and the best K for recall might be different.

In cancer detection, we'd want to tune K for Recall, not just Accuracy!

```

In [23]: # — SL-2: What happens if we DON'T scale? —————
# Question: We said scaling is important – let's actually prove it.

# Train KNN on UNSCALED data
knn_unscaled = KNeighborsClassifier(n_neighbors=best_k)
knn_unscaled.fit(X_train, y_train)
y_pred_unscaled = knn_unscaled.predict(X_test)

```

```

acc_scaled = accuracy_score(y_test, y_pred_knn)
acc_unscaled = accuracy_score(y_test, y_pred_unscaled)
rec_scaled = recall_score(y_test, y_pred_knn)
rec_unscaled = recall_score(y_test, y_pred_unscaled)

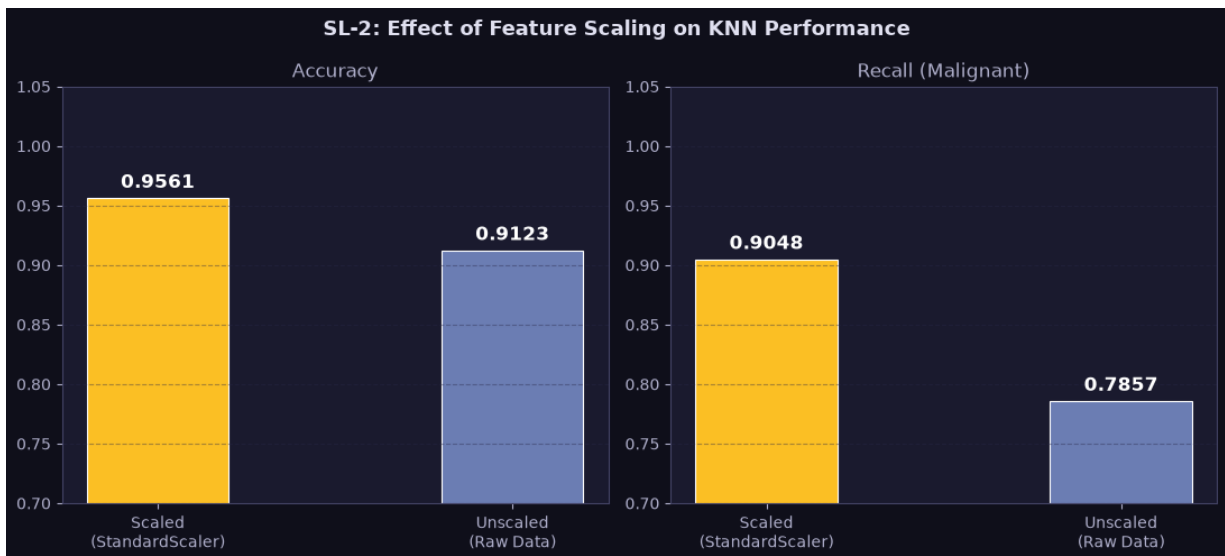
labels_bar = ['Scaled\n(StandardScaler)', 'Unscaled\n(Raw Data)']
fig, axes = plt.subplots(1, 2, figsize=(11, 5))
fig.suptitle('SL-2: Effect of Feature Scaling on KNN Performance',
             fontsize=13, fontweight='bold', color='#e0e0f0')

for ax, vals, title in zip(axes,
                           [[acc_scaled, acc_unscaled], [rec_scaled, rec_unscaled]],
                           ['Accuracy', 'Recall (Malignant)']):
    bars = ax.bar(labels_bar, vals, color=[COLORS['knn'], '#6b7db3'],
                  width=0.4, edgecolor='white', linewidth=0.8)
    for bar, val in zip(bars, vals):
        ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.005,
                f'{val:.4f}', ha='center', va='bottom', fontsize=12, color='white')
    ax.set_title(title, fontsize=12, color='white')
    ax.set_ylim(0.7, 1.05)
    ax.grid(axis='y', linestyle='--', alpha=0.4)

plt.tight_layout()
plt.savefig('sl2_scaling_effect.png', dpi=150, bbox_inches='tight', facecolor='black')
plt.show()

print(f"KNN Accuracy - Scaled: {acc_scaled:.4f} | Unscaled: {acc_unscaled:.4f}")
print(f"KNN Recall - Scaled: {rec_scaled:.4f} | Unscaled: {rec_unscaled:.4f}")
print()
print("Takeaway: This proves why we scaled! Unscaled KNN performs significantly worse.")

```



```

KNN Accuracy - Scaled: 0.9561 | Unscaled: 0.9123 | Diff: +0.0439
KNN Recall - Scaled: 0.9048 | Unscaled: 0.7857 | Diff: +0.1190

```

Takeaway: This proves why we scaled! Unscaled KNN performs significantly worse.

In [24]: # — SL-3: Effect of Regularization (C) on Logistic Regression —
 # Question: What if we tune the C parameter? How sensitive is LR to it?


```

C_values = [0.001, 0.01, 0.1, 1, 10, 100, 1000]
c_accs, c_recs, c_fls = [], [], []

for C in C_values:
    lr_c = LogisticRegression(C=C, random_state=42, max_iter=1000)
    lr_c.fit(X_train_scaled, y_train)
    preds = lr_c.predict(X_test_scaled)
    c_accs.append(accuracy_score(y_test, preds))
    c_recs.append(recall_score(y_test, preds))
    c_fls.append(f1_score(y_test, preds))

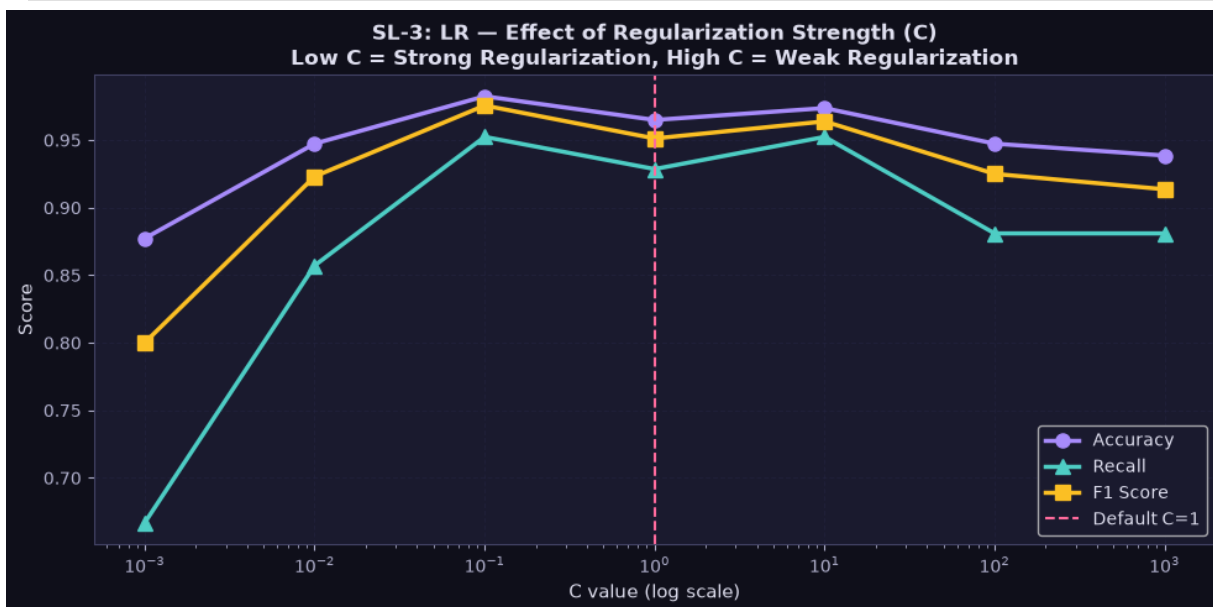
fig, ax = plt.subplots(figsize=(10, 5))
ax.semilogx(C_values, c_accs, marker='o', color=COLORS['lr'], linewidth=2.5)
ax.semilogx(C_values, c_recs, marker='^', color='#4ecdc4', linewidth=2.5)
ax.semilogx(C_values, c_fls, marker='s', color='#fbbf24', linewidth=2.5)
ax.axvline(x=1, color='#ff6b9d', linestyle='--', linewidth=1.5, label='Defau

ax.set_title('SL-3: LR – Effect of Regularization Strength (C)\nLow C = Stro
            fontsize=12, fontweight='bold', color='#e0e0f0')
ax.set_xlabel('C value (log scale)', fontsize=11)
ax.set_ylabel('Score', fontsize=11)
ax.legend(fontsize=10)
ax.grid(True, linestyle='--', alpha=0.4)

plt.tight_layout()
plt.savefig('sl3_lr_regularization.png', dpi=150, bbox_inches='tight', facec
plt.show()

print("Takeaway: Very low C → underfitting (model too simple, ignores data).
print("Very high C → overfitting (memorises training data, fails on test).")
print(f"Default C=1 is a good starting point. Best C here = {C_values[c_accs

```



Takeaway: Very low C → underfitting (model too simple, ignores data).
 Very high C → overfitting (memorises training data, fails on test).
 Default C=1 is a good starting point. Best C here = 0.1

```

In [25]: # — SL-4: KNN with Different Distance Metrics —————
# Question: KNN uses Euclidean distance by default – what if we try others?

from sklearn.model_selection import cross_val_score

distance_models = {
    'Euclidean (p=2)' : KNeighborsClassifier(n_neighbors=best_k, metric='min
    'Manhattan (p=1)' : KNeighborsClassifier(n_neighbors=best_k, metric='min
    'Chebyshev'       : KNeighborsClassifier(n_neighbors=best_k, metric='che
}

print(f"KNN (K={best_k}) with Different Distance Metrics – 5-Fold Cross Vali
print("-" * 55)
results = {}
for name, model in distance_models.items():
    scores = cross_val_score(model, X_train_scaled, y_train, cv=5, scoring='
    results[name] = scores.mean()
    print(f"{name:<20}: F1 = {scores.mean():.4f} ± {scores.std():.4f}")

fig, ax = plt.subplots(figsize=(9, 5))
bar_colors_sl = [COLORS['knn'], '#4ecdc4', '#ff6b9d']
bars = ax.bar(results.keys(), results.values(), color=bar_colors_sl,
               width=0.4, edgecolor='white', linewidth=0.8, alpha=0.9)
for bar, val in zip(bars, results.values()):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 0.002,
            f'{val:.4f}', ha='center', va='bottom', fontsize=11, color='whit

ax.set_title(f'SL-4: KNN (K={best_k}) – Distance Metric Comparison (5-Fold C
            fontsize=12, fontweight='bold', color='#e0e0f0')
ax.set_ylabel('Mean F1 Score', fontsize=11)
ax.set_ylim(min(results.values()) - 0.03, 1.01)
ax.grid(axis='y', linestyle='--', alpha=0.4)

plt.tight_layout()
plt.savefig('sl4_knn_distance_metrics.png', dpi=150, bbox_inches='tight', fa
plt.show()

print()
print("Takeaway: Euclidean and Manhattan are usually close in performance.")
print("Manhattan (L1) can sometimes be more robust when there are outliers.")
print("Chebyshev uses max-coordinate distance – works differently in high di

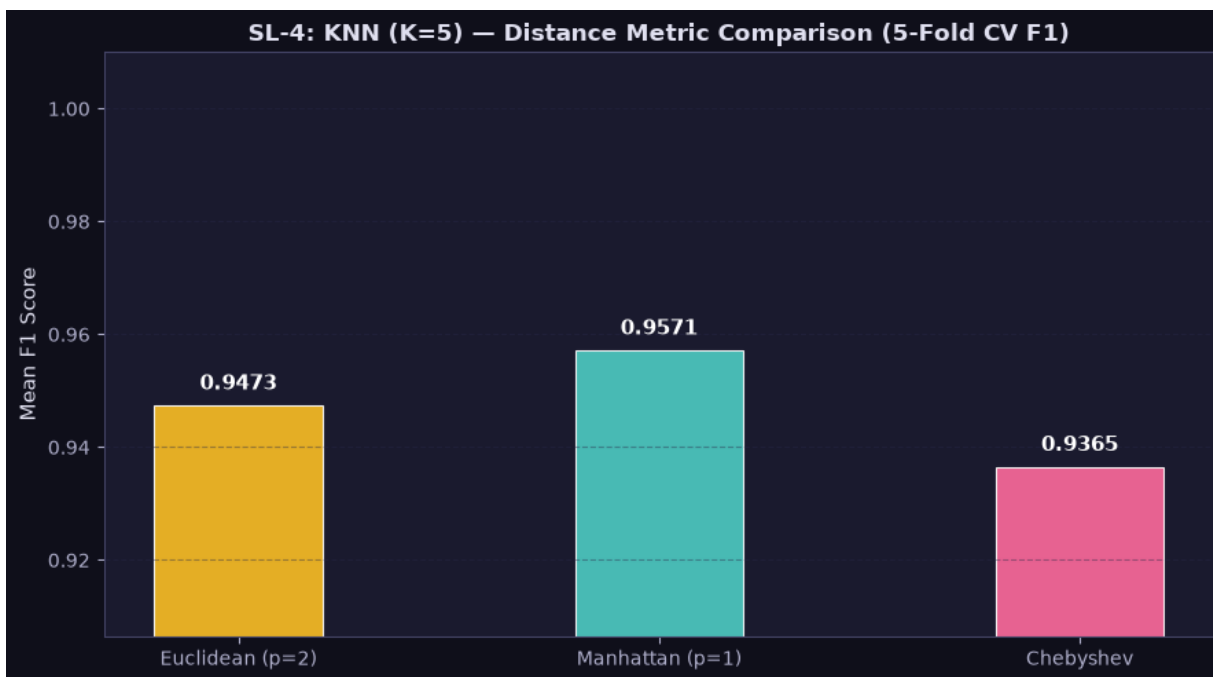
```

KNN (K=5) with Different Distance Metrics – 5-Fold Cross Validation:

```

-----
Euclidean (p=2)      : F1 = 0.9473 ± 0.0342
Manhattan (p=1)     : F1 = 0.9571 ± 0.0182
Chebyshev           : F1 = 0.9365 ± 0.0254

```



Takeaway: Euclidean and Manhattan are usually close in performance. Manhattan (L1) can sometimes be more robust when there are outliers. Chebyshev uses max-coordinate distance – works differently in high dimension s.

Self-Learning Takeaways

#	Question Asked	What We Found
SL-1	Is the best K for accuracy also the best for Recall?	Not always! For cancer detection, tune for Recall specifically.
SL-2	What if we skip scaling for KNN?	Performance drops significantly — this <i>proved</i> why we scaled.
SL-3	How sensitive is LR to the C parameter?	Very low C hurts accuracy. Default C=1 is safe. Higher C can sometimes help.
SL-4	Does the choice of distance metric in KNN matter?	Euclidean and Manhattan are usually close. Worth experimenting depending on data.

Lab 6 — Logistic Regression vs KNN | Breast Cancer Wisconsin Dataset